



Gymnázium J. K. Tyla

Úvod do agentického AI a harness pro automatizovaný softwarový vývoj

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

KONCEPT

Anotace

Tato odborná práce se zabývá principy agentního inženýrství (*agentic engineering*) a architekturou řídicích harnessů pro autonomní vývoj softwaru. Práce analyzuje fundamentální limity autoregresivních velkých jazykových modelů a představuje deterministické metody řízení jejich exekuce: formalizaci životního cyklu vývojových požadavků, bezpečnostní mantinely a detekci uvíznutí ve smyčce ReAct, správu kontextového okna a eliminaci sémantického posunu při kompresi historie, standardizované vyvolávání nástrojů a hierarchickou orchestraci subagentů. Cílem práce je formulovat ucelený architektonický rámec, který umožňuje spolehlivé a bezpečné zapojení autonomních agentů do reálného softwarového inženýrství pod deterministickým lidským dohledem.

Klíčová slova

agentní inženýrství, řídicí harness, autonomní systémy, softwarové inženýrství, orchestrace agentů, jazykové modely

Annotation

This thesis explores the foundational principles of agentic engineering and harness architecture for automated software development. The work analyzes the fundamental limitations of autoregressive large language models and presents deterministic execution control mechanisms: formalization of the engineering request lifecycle, runtime safety boundaries and stuck detection within the ReAct loop, context window management and semantic drift mitigation during compaction, standardized tool invocation, and hierarchical subagent orchestration. The objective is to formulate a robust architectural framework that enables reliable and secure integration of autonomous agents into real-world software engineering under deterministic human governance.

Keywords

agentic engineering, harness architecture, autonomous systems, software engineering, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	7
1.1 Motivace	7
1.2 Cíl práce	7
1.3 Metodika	8
2 Teoretická část	9
2.1 Řízení verzí	9
2.1.1 Větvě a jejich role	9
2.1.2 Model pull requestu	9
2.2 Kontinuální integrace	10
2.2.1 Požadované kontroly	10
2.2.2 Sestavení a artefakty	10
2.3 Jazykové modely a jejich orchestrace	10
2.3.1 Transformer a velké jazykové modely	11
2.3.2 Tokeny, tokenizér a embedding	11
2.3.3 Mechanismus pozornosti a KV cache	12
2.3.4 Multimodální modely	12
2.3.5 Context	12
2.3.6 Prompt a kontextové inženýrství	15
2.3.7 Agent vs Chatbot	16
2.3.8 Harness a System Prompt	17
2.3.9 ReAct smyčka	17
2.3.10 Vytváření nástrojů	19
2.3.11 Dovednosti, skripty a zachytané body	20
2.3.12 MCP Server	21
2.3.13 Meta Harness	21
2.3.14 Subagenti	21
2.3.15 Workflows neboli Graph Engineering	22
2.4 Human in the loop neboli člověk ve smyčce	23
2.4.1 Schvalovací body	23

2.4.2	Dohledatelnost	23
2.4.3	Eskalace a transparence selhání	23
3	Praktická část	24
3.1	Cíl a rozsah systému	24
3.2	Architektura	24
3.2.1	Koncepce a systémové vrstvy	24
3.2.2	Sdílení pracovních postupů	24
3.2.3	Konfigurační model repozitáře	24
3.3	Životní cyklus požadavku	25
3.4	Popis prostředí repozitáře	25
3.4.1	Domény a prostředí	25
3.4.2	Deklarace jako doplněk rozpoznávání	25
3.5	Orchestrace napříč poskytovateli	26
3.6	Ověřování změn	26
3.7	Projektová automatizace a správa nastavení jako kód	26
3.8	Automaticky generovaná dokumentace	27
3.9	Systém revizních značek pro lidský dohled nad akademickým textem	27
4	Výsledky a diskuse	29
4.1	Sjednocení pracovních postupů v architektuře harnessu	29
4.2	Přenositelnost napříč vývojovými doménami	29
4.3	Provozní metriky a spolehlivost agentních běhů	30
4.4	Poznatky a systémová úskalí z provozu	30
4.5	Diskuse: Porovnání architektur	30
4.6	Omezení	31
5	Závěr	32
	Seznam příloh	33
A	Obsah přiloženého média	34
B	Schéma konfiguračního manifestu darkfactory.json	35
C	Sdílené workflow pro GitHub Actions	36
D	Systémové prompty plánovacího a kódovacího agenta	37
E	Protokol revizních značek v sazebním systému Typst	38
E.1	Vizuální reprezentace jednotlivých prvků v sazbě	38
	Seznam zdrojů	40

Seznam obrázků a tabulek

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	18
Tabulka 1	Rozpoznávání prostředí podle souboru popisujícího balíček.	25

KONCEPT

1 Úvod

Vývoj softwaru se za posledních dvacet let z velké části zautomatizoval. Sestavení programu, spuštění testů, kontrola stylu i nasazení do provozu dnes obstarávají stroje a nikdo je nepovažuje za práci hodnou lidského času. Jeden krok však zůstal ruční: samotná změna zdrojového kódu. Právě tam se tvoří většina prodlev — mezi okamžikem, kdy někdo popíše požadavek, a okamžikem, kdy se změna dostane k uživatelům.

Výrobní průmysl zná pojem *dark factory*, tedy „temná továrna“: provoz, který běží bez lidské obsluhy, a proto v něm nemusí svítit. Nejde o představu úplného vyloučení člověka — i temná továrna má konstruktéry, kteří rozhodují, co se bude vyrábět — nýbrž o vyloučení člověka z opakujících se úkonů. Tato práce zkoumá, jak stejný princip uplatnit ve vývoji softwaru.

1.1 Motivace

S rozšířením velkých jazykových modelů se objevila možnost automatizovat i psaní kódu. Většina dostupných nástrojů však řeší jen dílčí krok: vygenerují návrh změny, který někdo musí zasadit do procesu, ověřit a schválit. Chybí popis toho, jak má takový nástroj zapadnout do vývojového procesu jako celku — kdo schvaluje, co se stane při selhání a jak se pozná, že je výsledek správný.

Právě tato mezera je předmětem práce. Zajímavá není otázka, zda model dokáže napsat kód; to je dnes doloženo. Zajímavá je otázka, jaké okolí musí kolem takového modelu vzniknout, aby jeho výstupu bylo možné důvěřovat.

1.2 Cíl práce

Cílem této práce je představit principy agentního inženýrství (*agentic engineering*) a navrhnout architekturu řídicího harnessu pro automatizovaný softwarový vývoj, který provede vývojový požadavek celým procesem od zadání po ověřenou změnu, aniž by se vzdal lidského dohledu v rozhodujících bodech.

Dílčí cíle:

1. Popsat současný stav automatizace vývoje softwaru a teoretická východiska agentních systémů.
2. Vymezit principy agentního inženýrství a deterministického řízení jazykových modelů (harness, správa kontextu, detekce uvíznutí v ReAct smyčce).
3. Navrhnout architekturu řídicího harnessu pro spolehlivý a bezpečný běh kódovacích agentů.
4. Analyzovat bezpečnostní mantinely, limity kontextu a provozní úskalí autonomních vývojových procesů.

V návaznosti na stanovené cíle si práce klade tři konkrétní inženýrské výzkumné otázky:

- **V01 (Míra automatizace a role člověka):** Lze vývojový proces od zadání požadavku (GitHub Issue) po vytvoření funkčního pull requestu strukturovat tak, aby role vývojáře spočívala v architektonickém dozoru a schvalování záměru (Human Gate), aniž by musel sám psát kód nebo řešit syntaktické regrese?
- **V02 (Řízení divergence a spolehlivost smyčky):** Jakými deterministickými mechanismy (stuck detection, rozpočet tahů, circuit breaker) lze v řídicím harnessu zabránit patologiím jazykových modelů, jako je perseverace, oscilace či nekonečné zacyklení v ReAct smyčce?
- **V03 (Integrita paměti a eliminace sémantického posunu):** Jakými postupy lze efektivně spravovat kontextové okno agenta při komplexních úlohách, aby nedocházelo k degradaci pozornosti (Context Rot) a destruktivní ztrátě architektonických invariantů při rekurzivní kompresi?

1.3 Metodika

Práce je z povahy tématu teoreticko-architektonická a inženýrská: primárním výstupem je formulace principů agentního inženýrství a návrh robustní architektury řídicího harnessu pro automatizovaný softwarový vývoj. Postup odpovídá inženýrskému cyklu:

1. **Koncepční analýza:** Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
2. **Architektonický návrh:** Formulace modulárního modelu řídicího harnessu zahrnujícího správu stavu, exekuční pískoviště (sandbox), bezpečnostní pojistky a orchestraci subagentů.
3. **Kritické zhodnocení:** Analýza navržených principů ve srovnání se současnými monolitickými agentními smyčkami a formulace provozních limitů autonomního inženýrství.

2 Teoretická část

Tato kapitola vymezuje pojmy, o které se opírá praktická část: řízení verzí, kontinuální integraci, orchestraci jazykových modelů a princip zapojení člověka do smyčky (*Human-in-the-loop*). Cílem není vyčerpávající přehled, nýbrž zavedení pojmů v podobě, v jaké s nimi pracuje navržený systém.

2.1 Řízení verzí

Systém pro řízení verzí uchovává historii změn zdrojového kódu. Distribuovaný model, jehož nejrozšířenějším zástupcem je Git, se od centralizovaného liší tím, že každý vývojář má úplnou kopii historie (1). Změny lze proto vytvářet a zkoumat i bez spojení se serverem a slučovat je až ve chvíli, kdy jsou hotové.

2.1.1 Větev a jejich role

V moderních distribuovaných systémech správy verzí (zejména v systému Git (1)) je repositář formálně modelován jako orientovaný acyklický graf (*Directed Acyclic Graph*, DAG), jehož uzly tvoří neměnné objekty revizí (*commits*) provázané kryptografickými hashy (SHA-1 či SHA-256) na své rodičovské stavy. Větev v tomto modelu nepředstavuje fyzickou kopii souborů, nýbrž odlehčený, pohyblivý ukazatel (*ref*) na konkrétní uzel grafu.

Tato grafová architektura má klíčový význam pro bezpečné zapojení autonomních agentů do vývojového procesu:

- **Izolace stavu:** Agent operuje ve vyhrazené větvi (např. `feature/...` nebo `agent/...`), která odbočuje z hlavní vývojové linie (`main`). Veškeré pokusné mutace souborového systému, mezistavy a ladící kroky zůstávají striktně izolované, aniž by ohrožily stabilitu produkčního kódu nebo práci ostatních členů týmu.
- **Deterministický audit:** Každý krok agenta lze reprezentovat jako atomický commit s přesným časovým otiskem, autorskými metadaty a odkazem na kontextové zadání. Vzniká tak neměnná a zpětně ověřitelná historie změn.
- **Strategie slučování:** Při integraci hotové větve do hlavní linie se uplatňují různé topologické strategie: přímý posun ukazatele (*fast-forward*), vytvoření explicitního slučovacího uzlu (*merge commit*), nebo sloučení celé sekvence dílčích mezikroků agenta do jediného čistého uzlu (*squash and merge*). Právě squashování je v autonomních pipeline preferováno, neboť eliminuje šum v podobě neúspěšných pokusů modelu a v hlavní větvi zanechává pouze finální, ověřený přírůstek.

2.1.2 Model pull requestu

Sloučení větve do hlavní vývojové linie se v moderním kolaborativním softwarovém inženýrství odehrává prostřednictvím modelu **pull requestu** (PR, na platformě GitLab též

Merge Request). Jde o formalizovaný procesní uzel, v němž autor větve předkládá navržený diff kódu k revizi dříve, než dojde k jeho trvalému začlenění do chráněné hlavní větve. V kontextu autonomního vývoje plní pull request dvě nezastupitelné funkce:

1. **Strojová validační brána:** Na vytvoření nebo aktualizaci PR reaguje integrační server (CI), který v izolovaném kontejneru spustí sadu automatizovaných testů, typových kontrol a bezpečnostních linterů. Tím je objektivně ověřeno, že syntetický kód generovaný modelem splňuje stanovené standardy a nezpůsobuje regresi stávající funkcionality.
2. **Lidská schvalovací brána (Human Gate):** Pull request poskytuje přehledné rozhraní zobrazující řádkový diff změn, výsledky automatických testů a strukturovaný popis úprav. Člověk v roli recenzenta (*code reviewer*) tak může provést finální sémantickou kontrolu a rozhodnout o schválení či zamítnutí změny. Pull request tím představuje ideální architektonický styčný bod pro princip člověka ve smyčce (*Human-in-the-loop*).

2.2 Kontinuální integrace

Kontinuální integrace (angl. *continuous integration*) je praxe, při níž se každá změna automaticky sestaví a otestuje (2). Namísto dlouhých období, kdy se změny hromadí a slučují až na konci, se ověřuje průběžně a v malých dávkách, takže chyba je odhalena blízko svému vzniku.

2.2.1 Požadované kontroly

Ke kontinuální integraci patří pojem *požadovaných kontrol* (angl. *required checks*): množina úloh, které musí skončit úspěšně, jinak nelze změnu sloučit. Tím se z kvality stává vlastnost vynucovaná strojem, nikoli pouze dohodou mezi vývojáři.

Návrh požadovaných kontrol má jedno nezřejmé úskalí. Úloha, která se za jistých okolností „přeskočí“, nehlásí žádný výsledek; je-li přitom uvedena mezi požadovanými, zablokuje slučování napořád. Kontrola proto musí vždy skončit nějakým závěrem — i kdyby jím bylo konstatování, že v daném repozitáři není co dělat.

2.2.2 Sestavení a artefakty

Výsledkem sestavení bývá *artefakt*: spustitelný soubor, knihovna, nebo — jak ukazuje praktická část — vysázený dokument. Automatizace vydávání verzí spojuje artefakt se značkou v historii, takže ke každé vydané verzi existuje doložitelný výstup.

2.3 Jazykové modely a jejich orchestrace

🔗 **Metodické vymezení / Rozsah práce:** Metodické vymezení a rozsah práce: Těžištěm této práce není strojové učení, matematická optimalizace vah ani trénování neuronových sítí. Jazykový model vnímáme jako hotovou inferenční komponentu vystupující v roli stochastického kognitivního jádra. Ústředním předmětem zkoumání je **agentní inženýrství** (*agentic engineering*) a **architektura řídicího harnessu** pro autonomní vývoj softwaru. Následující sekce věnované transformeru,

tokenizeru, embeddingu a pozornosti jsou proto záměrně zredukovány na nezbytné konceptuální minimum potřebné pro pochopení kontextového okna, spotřeby tokenů a fungování rozhraní nástrojů.

2.3.1 Transformer a velké jazykové modely

Základním stavebním kamenem moderních kódovacích agentů jsou velké jazykové modely (*Large Language Models*, LLM) vystavěné na architektuře dekodérového transformru (*Decoder-only*, např. GPT, Claude, LLaMA či DeepSeek) (3). Na rozdíl od starších sekvencních architektur (RNN) zpracovává transformer vstupní data pomocí mechanismu pozornosti (*attention*), který umožňuje modelu paralelně zohledňovat vztahy mezi všemi prvky v zadaném kontextu bez ohledu na jejich vzájemnou vzdálenost. V agentním inženýrství model nevystupuje jako magický vševědoucí systém, nýbrž jako **stochastický autoregresivní generátor**: na základě zadané textové historie (kontextu) opakovaně predikuje nejpravděpodobnější následující symboly. Tyto symboly mohou reprezentovat jak přirozený jazyk či programovací kód, tak strukturované instrukce pro volání externích nástrojů.

2.3.2 Tokeny, tokenizer a embedding

Jazykový model neoperuje přímo se slovy či znaky, nýbrž s diskrétními jednotkami zvanými **tokeny**. Převod mezi surovým textem a číselnými identifikátory tokenů zajišťuje **tokenizer** — deterministický program pracující s pevným slovníkem dílčích slovních podjednotek (*sub-words*, např. algoritmem Byte-Pair Encoding). Běžná anglická slova odpovídají zpravidla jednomu tokenu (cca 3–4 znaky), zatímco méně časté výrazy, technické identifikátory nebo jazyky s bohatou flexí a diakritikou (včetně češtiny) jsou děleny do vícero tokenů. Před samotným výpočtem neuronové sítě je každý token převeden do vícerozměrného vektorového prostoru — tzv. **embeddingu**. Vektorové vnoření přiřazuje slovům geometrické souřadnice tak, aby sémanticky příbuzné koncepty (např. volání funkce a její definice) ležely v prostoru blízko sebe.

Z pohledu agentního inženýrství mají vlastnosti tokenizace tři zásadní provozní dopady:

1. **Kapacitní strop kontextového okna:** Každý model má pevně stanovený maximální počet tokenů, které dokáže v jednom běhu pojmout.
2. **Ekonomické náklady a latence:** Výpočetní náročnost i cena komerčních API se účtují za počet zpracovaných a vygenerovaných tokenů.
3. **Jazyková asymetrie:** Texty v češtině spotřebovávají 2× až 3× více tokenů než ekvivalentní sdělení v angličtině. V autonomních pipeline je proto optimální udržovat vnitřní systémové prompty, technické plány a strukturované logy v angličtině.

2.3.3 Mechanismus pozornosti a KV cache

Mechanismus pozornosti (*Attention*) umožňuje transformeru dynamicky propojovat informace napříč celou historií sezení — například spojit chybový výpis kompilátoru z konce kontextu s deklarací proměnné na jeho začátku.

Z výpočetního hlediska vyžaduje pozornost porovnání každého tokenu se všemi ostatními tokeny v sekvenci, což vede ke kvadratické výpočetní a paměťové složitosti $O(N^2)$ vzhledem k délce kontextu N . Aby nebylo nutné při každém vygenerovaném slově přepočítávat celou historii od začátku, ukládají inferenční enginy mezivýpočty do mezipaměti — tzv. **KV cache** (*Key-Value Cache*). Velikost a správa KV cache představují primární hardwarový limit pro délku kontextového okna a zásadní faktor pro rychlost odezvy agenta.

2.3.4 Multimodální modely

Ačkoliv byl transformer původně navržen pro text, stejný princip lze aplikovat i na obrazové vstupy (*Vision-Language Models*, VLM). Obraz je rozložen na pravidelnou mřížku dlaždic (*patches*), které jsou lineární projekcí transformovány na vizuální tokeny se stejnou dimenzí vnoření jako tokeny textové. Mechanismus pozornosti pak operuje společně nad textem i obrazem.

Ačkoli multimodální modely nabízejí teoretický potenciál pro doplňkové softwarové úlohy (např. interpretaci grafických wireframů v zadání nebo vizuální kontrolu webového rozvržení), v agentním inženýrství zůstávají autonomní kódovací systémy primárně textové — staví na práci se zdrojovým kódem, abstraktními syntaktickými stromy (AST), unifikovanými diffy v Gitu a textovými protokoly testovacích nástrojů.

2.3.5 Context

2.3.5.1 Turn

Každému diskrétnímu kroku ve výměně informací mezi okolním prostředím a jazykovým modelem se v agentních architekturách říká tah (*turn*). Na rozdíl od jednoduchého konverzačního rozhraní, kde dochází pouze ke střídání uživatele a asistenta, agentní smyčka (typicky implementující vzor ReAct (4)) rozlišuje tři základní typy tahů:

1. **Tah uživatele či prostředí (*User / Environment Turn*):** Vnáší do kontextu nové zadání, externí událost (např. spuštění GitHub webhooku) nebo doplňující instrukce.
2. **Tah modelu (*Model / Assistant Turn*):** Reprezentuje jedno spuštění inference nové sítě. Model na základě dosavadní historie emituje buď finální textovou odpověď, nebo strukturovaný požadavek na vyvolání nástroje (*tool call*).
3. **Tah vykonání nástroje (*Tool Execution Turn*):** Běhové prostředí (*harness*) provede požadovanou operaci — např. spuštění skriptu v terminálu či čtení souboru — a její výsledek vloží do kontextu jako syntetický tah určený pro navazující uvažování modelu.

Striktní oddělení těchto fází a jejich deterministická serializace do historie kontextu mají zásadní dopad na výpočetní efektivitu: umožňují inferenčnímu serveru plně využívat cachování klíčů a hodnot (*KV cache*), neboť neměnná historie předchozích tahů nemusí být při každém kroku znovu přepočítávána.

2.3.5.2 Context Window

Model je schopen přijmout pouze omezený objem vstupu; tomuto limitu se říká kontextové okno (*Context Window*) a vyjadřuje se v počtu tokenů. Na rozdíl od slovníku (*vocabulary*), který vymezuje pouze repertoár známých tokenů, je maximální délka kontextového okna určena architekturou pozičního kódování (např. škálováním frekvenčních bází v RoPE) a hardwarovou náročností mechanismu pozornosti — kvadratickou složitostí $O(N^2)$ a velikostí alokované KV cache v operační či grafické paměti.

2.3.5.3 Compaction

Představuje proces sumarizace a zkrácení historie, který řídicí harness iniciuje ve chvíli, kdy zaplnění kontextového okna dosáhne stanoveného prahu. Zpravidla jde o vyvolání modelu se specifickou systémovou instrukcí pro bezztrátovou syntézu dosavadního průběhu sezení a kompletním protokolem dosavadní komunikace. Zpravidla jde o vyvolání modelu se specifickou systémovou instrukcí k syntéze dosavadního průběhu sezení a kompletním protokolem dosavadní komunikace. Výsledný zkrácený kontext následně v kontextovém okně nahradí původní rozsáhlou historii kroků.

2.3.5.3.1 Limity rekurzivní komprese a sémantický posun (Semantic Drift)

Ačkoliv je proces komprese historie v řadě základních implementací prezentován jako přímočaré řešení konečné kapacity kontextového okna, z hlediska teorie informace a systémového inženýrství představuje **destruktivní ztrátovou kompresi**. Při zkracování dosavadní trajektorie model trpí výrazným **konfirmačním zkreslením** (*confirmation bias*): autoregresivní model při generování souhrnu nevybírá fakta nestranně, nýbrž sumarizuje primárně ty aspekty historie, které sám považuje za relevantní vzhledem ke svým vnitřním statistickým asociacím.

Tato ztrátovost má fatální dopad na integritu vývojového kontextu:

1. **Vymazání deterministických detailů:** Zkrácením nevratně zanikají přesná čísla řádků, signatury privátních funkcí, plné cesty souborů a doslovná znění chybových hlášení kompilátoru či testovacího frameworku (*stack traces*).
2. **Ztráta negativních omezujících podmínek:** Zvláště náchylné k vymizení jsou explicitní negativní mantinely stanovené uživatelem v úvodu úlohy (např. „neměň veřejné rozhraní modulu X“ nebo „nepřidávej žádné externí závislosti“). V syntetizovaném

souhrnu bývají tyto kritické podmínky zobecněny či zcela opomenuty, což v navazujících tazích vede k jejich okamžitému porušení.

3. **Sémantický posun (Semantic Drift):** Při dlouhotrvajících komplexních úlohách vyžadujících vícenásobnou rekurzivní kompresi dochází ke kaskádovému kumulování chyb (analogii hry na tichou poštu). Pokud je stav S_{k+1} generován jako souhrn předchozího souhrnu S_k a nového přírůstku Δ_k , drobné aproximační odchylky či halucinace z kola k jsou v kole $k + 1$ přijaty jako nezvratná historická fakta. Po několika cyklech komprese se vnitřní model reality agenta zcela rozejde se skutečným stavem repozitáře a původním zadáním.

K překonání fundamentálních limitů textové rekurzivní komprese se v moderních agentních architekturách uplatňují tři rigorózní alternativy:

- **Hierarchická epizodická paměť a vyhledávání (RAG):** Kompletní doslovná historie tahů, výpisů nástrojů a diffů se nezkracuje přímo v kontextu, nýbrž persistuje do externí databáze (např. fulltextového či vektorového indexu). Do aktivního kontextu jsou následně deterministicky či sémanticky injektovány pouze ty minulé fragmenty, které jsou bezprostředně relevantní pro řešený podúkol.
- **Persistentní graf stavu projektu (Project State Graph):** Namísto nestrukturovaného volného textu udržuje harness explicitní strukturovaný stav projektu (např. formou formálního stavového schématu či grafového modelu), který deterministicky eviduje modifikované soubory, seznam nezodpovězených otázek, stav integračních testů a neměnitelné invarianty. Aktualizace tohoto stavu se řídí přísným schématem a nepodléhá volné narativní degradaci.
- **Selektivní prořezávání KV cache (Selective KV Cache Eviction):** Místo manipulace s textovými tokeny probíhá řízení paměti přímo na úrovni tenzorů v inferenčním enginu. Algoritmy jako StreamingLLM (5) či H_2O (Heavy Hitter Oracle) (6) identifikují a v paměti trvale uchovávají tzv. kotevní tokeny pozornosti (*attention sinks*) a matematicky nejvýznamnější minulé stavy, zatímco redundantní mezilehlé stavy bezpečně uvolňují bez nutnosti destruktivního přepisování promptu.

2.3.5.4 Context Rot a evaluace vybavování (Needle In A Haystack)

Označuje empiricky zdokumentovanou degradaci schopnosti modelu věnovat rovnoměrnou pozornost všem částem historie (tzv. jev *Lost in the Middle* (7)). Čím plnější je kontextové okno, tím méně jsou modely schopny spolehlivě vybavovat jemné detaily z úvodu sezení a dodržovat negativní omezující podmínky zadání.

Ke standardizovanému testování schopnosti modelu přesně vybavovat informace z rozsáhlého kontextového okna slouží syntetický evaluační test **Needle In A Haystack** (NIAH – „jehla v kupce sena“). Princip spočívá ve vložení izolovaného, na kontextu nezávislého faktu

(např. unikátního klíče či specifického nastavení konfigurace) do různé relativní hloubky (od 0 % do 100 %) dlouhého distraktoru textu (čítajícího desítky tisíc až miliony tokenů). Model je následně vyzván k zodpovězení dotazu, jehož vyřešení závisí výhradně na této vložené informaci.

Zatímco moderní frontier modely dosahují na jednoduchém syntetickém testu NIAH téměř stoprocentní úspěšnosti napříč celou délkou okna, v reálném softwarovém vývoji vyvstává zásadní omezení: práce v repozitáři vyžaduje tzv. **Multi-Needle Reasoning** — schopnost současně nalézt, propojit a logicky syntetizovat několik vzájemně závislých informací (např. signaturu funkce v jednom modulu, její volání v druhém a konfigurační flag ve třetím). V takovém scénáři se naplno projevuje degradace pozornosti: s rostoucím objemem historie v kontextovém okně rapidně klesá spolehlivost křížového uvažování, což v dlouhých agentních sezeních vede k přehlédnutí okrajových podmínek a regresi v kódu.

2.3.5.5 KV Caching

KV caching slouží ke snížení výpočetní náročnosti modelu při generování tokenů. Místo toho, abychom při každém kroku inference znovu od začátku počítali pozornost pro celou dosavadní konverzaci, ponechává inferenční engine v paměti uložené již vypočtené vektory klíčů a hodnot (*Key-Value pairs*) a v každém kroku počítá a přidává pouze nově vygenerovaný token.

2.3.6 Prompt a kontextové inženýrství

Prompt engineering (inženýrství promptů) je disciplína zaměřená na systematický návrh, formulaci a optimalizaci textových instrukcí předkládaných modelu (8). V autonomních agentních systémech nepředstavuje prompt pouhou volnou konverzaci, ale slouží jako závazný kontrakt vymezující chování, práva a bezpečnostní mantinely agenta. Mezi klíčové techniky patří:

- **Systémový prompt (System Prompt):** Základní direktiva definující identitu agenta, dostupné nástroje a striktní provozní pravidla (např. pravidla pro zachování neměnnosti existujících testů, konvence pro formát commitů či zákaz destruktivních operací v repozitáři).
- **Few-shot prompting:** Technika vložení několika vzorových příkladů (vstup-výstup) přímo do kontextu. Tím model získá konkrétní představu o požadovaném schématu (např. syntaxi konfiguračního souboru `darkfactory.json`) bez nutnosti dodatečného dotrénování parametrů.
- **Chain-of-Thought (myšlenkový řetězec):** Vedení modelu k explicitní formulaci mezikroků a vnitřní dedukce před vygenerováním konečného kódu či akce. Rozklad komplexního zadání na postupné logické kroky zásadně potlačuje halucinace a tvoří základ fáze rozvahy (*Thought*) v cyklu ReAct.

- **Injekce dynamického kontextu:** Průběžné doplňování promptu o aktuální stav repozitáře, stromovou strukturu souborů, chybové výpisy kompilátoru a výsledky testů, díky čemuž agent operuje nad reálnými fakty namísto odhadů.

2.3.6.1 Úskalí negativních instrukcí a princip afirmativního vymezení

Zvláštní výzvou při formulaci systémových pravidel a zadání úkolů je definice omezujících podmínek pomocí zákazů a negativních instrukcí (např. „nikdy nemaž existující testy“, „neupravuj soubory mimo složku src“). V praxi je empiricky prokázáno, že jazykové modely negativní příkazy často porušují nebo zcela ignorují.

Tento jev má hluboké opodstatnění v samotném matematickém fungování mechanismu pozornosti transformeru:

1. **Pozornost zaměřená na přítomná slova:** Matice pozornosti (QK^T) vyhodnocuje sémantickou relevanci na základě **přítomných** tokenů. Zmínka zakázané operace (např. `test_suite.py` ve větě „Za žádných okolností nemaž test_suite.py“) aktivuje v embeddingovém prostoru silné vektorové asociace k danému souboru i k operaci smazání.
2. **Asymetrie negace v autoregresivní predikci:** Negace představuje logický operátor vysokého řádu vyžadující složenou kompozici významu. Při autoregresivním generování token po tokenu však model často podlehne silnější statistické asociaci mezi přítomnými sémantickými pojmy dříve, než uplatní logický kontext záporové částice.

V robustním inženýrství agentních systémů se proto uplatňují dva komplementární principy:

- **Afirmativní formulace pravidel:** Pravidla se namísto zákazů formulují pozitivním vymezením požadovaného chování a mantinelů (např. „Upravuj výhradně soubory deklarované v seznamu `target_files`“, „Před jakoukoli změnou ověř běh stávajících testů“).
- **Deterministická exekuční ochrana v harnessu:** Bezpečnostní mantinely a zákazy destruktivních operací nesmí záviset na poslušnosti jazykového modelu vůči promptu. Řídící harness systému DarkFactory proto prosazuje striktní omezení deterministicky na úrovni běhového prostředí — např. připojením chráněných cest v souborovém systému pouze pro čtení (*read-only mount*), sandboxováním exekuce terminálových příkazů a nezávislou validací změn v pull requestu před sloučením.

2.3.7 Agent vs Chatbot

Fundamentální rozdíl mezi chatbotem a agentem spočívá v rozsahu interakce s okolním světem: zatímco chatbot operuje výhradně v uzavřeném konverzačním rozhraní a generuje textové odpovědi, agentem se systém stává ve chvíli, kdy je vybaven sadou výkonných nástrojů (*tools*). Prostřednictvím nich dokáže aktivně zkoumat repozitář, vyhledávat informace na webu, modifikovat soubory a autonomně vykonávat příkazy v cílovém výpočetním prostředí.

2.3.8 Harness a System Prompt

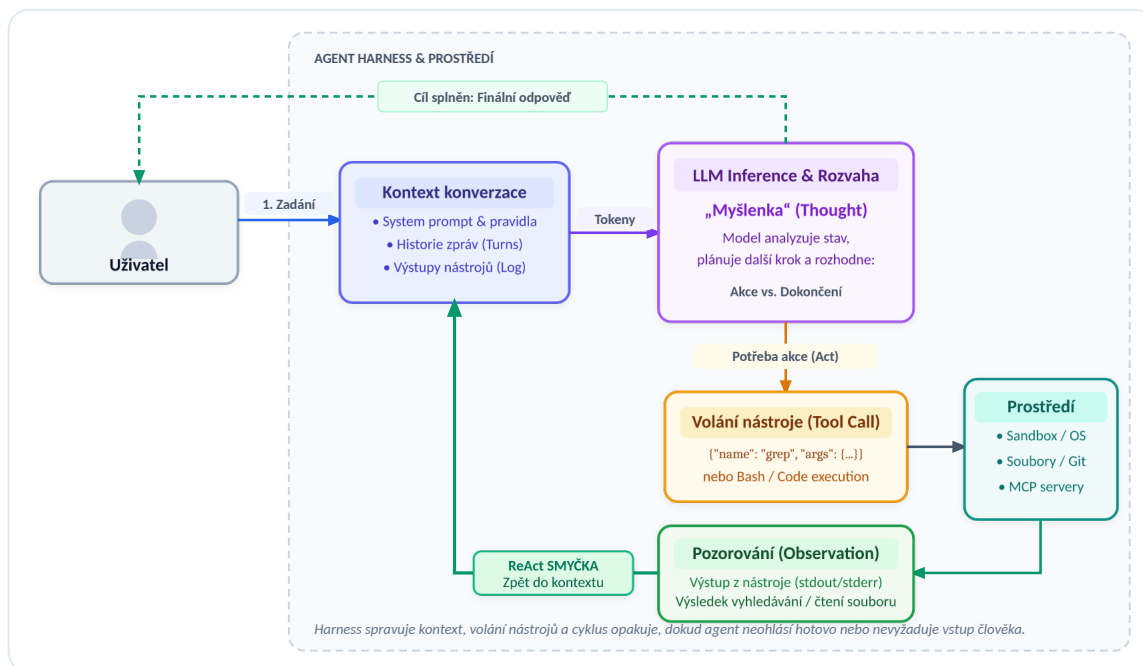
Harness je řídicí program obklopující jazykový model. Odlišuje se od inferenčního jádra (*inference engine*), které provádí samotné maticové výpočty sítě: harness má na starost rozhraní mezi uživatelem a agentem či chatbotem, předávání systémového promptu, bezpečné spouštění nástrojů a řízení iterativní smyčky neboli ReAct smyčky. Mezi typické příklady patří webová aplikace ChatGPT, terminálové rozhraní Claude Code, desktopová aplikace Codex a další agentní prostředí.

2.3.9 ReAct smyčka

Smyčka ReAct (*Reasoning and Acting*) tvoří základní stavební kámen, který z jazykového modelu vytváří autonomního agenta namísto pouhého pasivního generátoru textu (4). Princip nespočívá v jednorázovém vygenerování kódu či odpovědi, nýbrž v soustavné alter-naci vnitřní rozvahy (*Reasoning* neboli *Thought*) a vnějšího jednání (*Acting* neboli *Tool Call*). Tento princip byl formálně zaveden v práci *ReAct: Synergizing Reasoning and Acting in Language Models* ((4), [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)).

Jak znázorňuje diagram na Obrázek 1, po přijetí uživatelského zadání dochází k následujícímu iterativnímu cyklu:

1. **Fáze rozvahy (*Thought*):** Model v rámci inference analyzuje aktuální stav kontextu a formuluje hypotézu či myšlenkový postup, na jehož základě určí nejbližší nezbytný krok.
2. **Vyvolání akce (*Tool Call*):** Pokud krok vyžaduje externí operaci, model emituje strukturovaný požadavek na volání nástroje.
3. **Vykonání v harnessu a pozorování (*Observation*):** Řídicí harness zachytí požadavek, bezpečně jej provede v cílovém prostředí (terminál, souborový systém či MCP server) a vrácený výstup připojí do kontextového okna jako nové pozorování.
4. **Navazující iterace:** Aktualizovaný kontext je předložen modelu v dalším tahu, čímž agent adaptivně reaguje na reálnou odezvu prostředí namísto slepé generace.



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

2.3.9.1 Řízení divergence, detekce uvíznutí a rozpočet tahů (Step Budget)

Základní teoretický popis smyčky ReAct předpokládá, že proces přirozeně konverguje k vyřešení problému nebo k rozhodnutí modelu o předání finální odpovědi uživateli. V reálné autonomní softwarové praxi je však tento předpoklad zásadně lichý: jazykový model je stochastický systém bez inherentních garancí terminace a při řešení neúplně specifikovaných či složitých chyb v repozitáři často podléhá patologickým stavům divergence:

1. **Perseverace a zacyklení (*Perseveration & Infinite Loops*)**: Model opakovaně emituje identické volání nástroje se zcela shodnými parametry (např. čte tentýž neexistující soubor, provádí tentýž neúspěšný regex match nebo spouští tentýž selhávající test), a to i přesto, že z prostředí obdržel chybovou odezvu (*Observation*). Z hlediska pravděpodobnostní dynamiky dochází ke vzniku tzv. **atraktoru v kontextu**: přítomnost chybujiícího pokusu a chybové hlášky v bezprostřední historii paradoxně zvyšuje podmíněnou pravděpodobnost, že model v dalším tahu vygeneruje velmi podobný tokenový sled.
2. **Oscilace a těkání (*Thrashing*)**: Model střídavě přepíná mezi dvěma protichůdnými zásahy (např. úprava funkce A vyvolá regresi v modulu B, model následně zedituje modul B a rozbije modul A, načež změny cyklicky invertuje bez hlubšího pochopení systémové příčiny).
3. **Nekontrolovaná spotřeba zdrojů (*Context Runaway*)**: Bez striktního vnějšího omezení může divergentní smyčka v krátkém čase zcela zaplnit kontextové okno a vyčerpat finanční rozpočet na API volání.

Z těchto důvodů nesmí být rozhodnutí o ukončení cyklu ponecháno výhradně na autonomní vůli modelu. Řídící harness musí vůči modelu vystupovat jako deterministický dozorčí mechanismus (*watchdog*) a prosazovat následující vrstvy ochrany:

- **Rozpočet tahů a nákladů (*Step & Cost Budget*):** Každému sezení či dílčímu podúkolu je přiřazen pevný rozpočet maximálního počtu tahů $T_{\max}$ (typicky 25–50 kroků) a finanční strop pro spotřebu tokenů. Překročení tohoto rozpočtu vede k okamžitému a bezpodmínečnému zastavení inference.
- **Algoritmická detekce uvíznutí (*Stuck Detection*):** Běhové prostředí počítá pro každé volání nástroje kanonický hash jeho identifikátoru a normalizovaných argumentů:

$$h_t = \text{hash}(\text{nástroj}, \text{canonicalize}(\text{argumenty}))$$

Harness v klouzavém okně posledních k tahů (např. $k = 3$) detekuje identické volání $h_t = h_{t-1} = \dots = h_{t-k+1}$ nebo periodické vzorce v posloupnosti hashů.

- **Dvoustupňová intervenční strategie:**
 1. *Měkká intervence (Synthetic Warning):* Při první detekci repetice harness přeruší standardní tok a injektuje do kontextu syntetickou systémovou observaci s vysokou prioritou (např. „Varování harnessu: Nástroj byl opakovaně vyvolán s identickými argumenty bez pozitivního posunu. Přehodnoťte hypotézu, změňte diagnostický přístup nebo požádejte o doplňující instrukce.“). Tím se účinně rozbije pravděpodobnostní atraktor v matici pozornosti.
 2. *Tvrdá intervence (Circuit Breaker):* Pokud perseverace pokračuje i po varování, harness cyklus okamžitě ukončí, provede deterministický návrat (*rollback*) neuložených změn v repozitáři na poslední stabilní gitový commit a eskaluje stav lidskému operátorovi jako neřešitelnou anomálii.

2.3.10 Vyvolávání nástrojů

Abychom z jazykového modelu vytvořili autonomního agenta, musíme jej vybavit rozhraním pro interakci s okolním prostředím — tzv. nástroji (*tools*). V praxi existují dva převažující přístupy k realizaci nástrojů:

1. **Strukturované volání nástrojů (Function / Tool Calling v JSON):** Model ve svém výstupu zanechá strukturované volání odpovídající schématu zadanému v definici nástroje. Řídící harness toto volání zachytí, vykoná požadovanou funkci a vrátí výsledek zpět agentovi opět jako strukturovaná data. Tato možnost je spolehlivá pro jednoúčelové operace (např. integraci do podnikových systémů či zákaznické podpory — angl. *customer support*). Pro komplexní softwarový vývoj však představuje nevýhodu vysoká režie formátu JSON: velké množství formátovacích znaků se převádí na tokeny, což zbytečně plní kontextové okno a může vést k degradaci pozornosti (*Context Rot*).

2. **Přímé spouštění kódu (Code Execution):** Druhou možností je poskytnout agentovi plnohodnotné běhové prostředí (např. Bash, Python či TypeScript). Agent vygeneruje kód pro splnění svého záměru a harness jej spustí buď po jednotlivých příkazech, nebo jako ucelený skript. Z bezpečnostních důvodů je nezbytné, aby agent nepracoval přímo na nechráněném hostitelském počítači, nýbrž ve virtuálním prostředí odděleném bezpečnostní vrstvou — tzv. sandboxu (pískovišti). Tento způsob je pro softwarový vývoj nejefektivnější.

 **Hlubková kritika / Oponentura:** Iluzorní bezpečnost pískoviště: Druhý přístup (přímé spouštění kódu v kontejneru) práce nekriticky prezentuje jako „nejefektivnější“, zamlčuje však gigantická bezpečnostní a operační rizika. Běžný Docker kontejner není plnohodnotná bezpečnostní hranice (*security boundary*). Spuštění netestovaného kódu generovaného modelem s přístupem k síti otevírá prostor pro útoky typu Server-Side Request Forgery (SSRF), úniky tajných environmentálních proměnných (GitHub tokeny, API klíče k LLM poskytovatelům) přes postranní síťové kanály a nevratné poškození repozitáře. Práce postrádá jakoukoli analýzu formální izolace (např. microVM architektury jako AWS Firecracker, gVisor či striktní network namespaces) a nezodpovídá otázku, co zabráni kompromitovanému modelu zneužít CI runner jako odrazový můstek pro kompromitaci infrastruktury.

2.3.11 Dovednosti, skripty a záchytné body

Tyto standardy vznikly proto, aby vývojáři mohli modulárně upravovat chování agenta a rozšiřovat jeho schopnosti pro specifické doménové úlohy:

- **Dovednost (Skill):** Samostatný adresář sdružující instrukce, reference a pomocné soubory. Klíčovým prvkem je soubor `SKILL.md`, který využívá hlavičku v metadatovém formátu YAML frontmatter (ohraňovanou trojicí pomlček `---`). V hlavičce je definován název a stručný popis dovednosti. Řídící harness do základního systémového promptu vkládá pouze tato metadata; samotný text podrobného návodu se do kontextu načte až ve chvíli, kdy agent danou dovednost vyvolá. Tím se efektivně šetří kapacita kontextového okna.
- **Skript (Script):** Jednoúčelový spustitelný program (např. v jazyce Python či Bash). Skripty jsou pro efektivitu práce vitální: agent nemusí generovat každý příkaz z paměti, ale spouští deterministické a otestované postupy pro analýzu a transformaci kódu.
- **Záchytný bod (Hook):** Skript, který se v harnessu automaticky vyvolá při určité systémové události či stavovém přechodu (např. při inicializaci sezení, před odesláním promptu modelu nebo při selhání nástroje). Hooky umožňují vynucovat bezpečnostní pravidla a logování nezávisle na vůli samotného modelu.

2.3.12 MCP Server

Model Context Protocol vznikl v laboratořích společnosti Anthropic jako standardizovaný způsob pro efektivní interakci agentů s API servery. Ve své podstatě se jedná o nástroje postavené na rozhraní externího API, tudíž protokol sjednocuje různé standardy do jediného otevřeného rozhraní a přidává kontext — metadata pro každou přítomnou metodu, aby agent předem věděl, co od ní očekávat, a nemusel sám odhadovat fungování backendu. Praktický význam specifikace Model Context Protocol (9) spočívá ve sjednocení dříve fragmentovaného ekosystému proprietárních rozhraní a jednoúčelových integračních skriptů. Řídící harness se díky standardu stává univerzálním klientem a veškeré externí schopnosti, datové zdroje či nástroje se integrují jako samostatné procesy — tzv. MCP servery. Komunikace probíhá přes standardizovaný protokol JSON-RPC, a to buď lokálně prostřednictvím standardního vstupu a výstupu (`stdio`), nebo vzdáleně pomocí protokolu HTTP se Server-Sent Events (`SSE`). Tento modulární přístup striktně odděluje běhové prostředí agenta od samotné implementace nástrojů: libovolnou schopnost (např. přístup k databázi, vyhledávání v repozitáři nebo správu GitHub úkolů) stačí naimplementovat jednou a lze ji okamžitě zpřístupnit jakémukoli kompatibilnímu agentovi bez nutnosti zásahu do kódu samotného harnessu.

2.3.13 Meta Harness

V reálném světě se ukázalo, že nejefektivnější harness je takový, který dá samotnému agentovi rozhodovací pravomoc nad vlastní architekturou a umožňuje kontinuální změnu, tedy samořízenou evoluci (10). Tento přístup je mimořádně efektivní pro maximalizování pracovní kapacity agenta, v praxi se však hodí především pro osobního agenta. U komerčních nasazení, jako je zákaznická podpora (*customer support*), provozovatel naopak striktně vyžaduje deterministické mantinely a vylučuje, aby model modifikoval své vlastní provozní instrukce.

2.3.14 Subagenti

Když je agent vybaven nástrojem umožňujícím vyvolat další specializovanou instanci jazykového modelu, zadat jí dílčí úlohu, asynchronně s ní komunikovat a sledovat její postup, efektivita řešení rozsáhlých softwarových problémů dramaticky roste. V teorii autonomních systémů se tyto delegované entity označují jako subagenti (*subagents*). Tato architektura umožňuje hierarchickou dekompozici problému: hlavní koordinační agent (*orchestrator*) udržuje globální strategii a pověřuje úzce profilované subagenty izolovanými činnostmi — například řešerší dokumentace, prozkoumáním rozsáhlého adresářového stromu repozitáře nebo syntaktickou opravou konkrétního modulu. Zásadní architektonickou výhodou je ochrana a izolace kontextového okna: rozsáhlý a výpočetně náročný průzkumný kontext subagenta se po dokončení úkolu zahodí a rodičovskému

orchestrátoru je předán pouze syntetizovaný, čistý výsledek. Tím se zabráňuje zahlcení primárního kontextu (*context pollution*) a degradaci kognitivních schopností hlavního agenta.

2.3.15 Workflows neboli Graph Engineering

Jakmile se snažíme organizovat nebo automatizovat úlohu složitější než několik přímochrých kroků, lineární sekvence promptů přestává stačit. V takovém okamžiku přicházejí na řadu pracovní postupy (*workflows*), v teoretické informatice reprezentované jako grafy. Grafy se v softwarovém vývoji používají v mnoha situacích. Graf se skládá z uzlů (*nodes*) a hran (*edges*) — v kontextu agentních systémů každý uzel představuje izolovaného agenta s připraveným systémovým promptem, dovednostmi a nástroji, zatímco spoje mezi nimi vymezují tok informací a posloupnost práce. Abstrakcí tohoto grafového systému jsme schopni tuto pipeline nasadit na libovolnou úlohu, a to i dynamicky za pomoci orchestrace nadřazeným agentem.

Tento koncept se v informatice označuje jako orientovaný acyklický graf (**DAG** — *Directed Acyclic Graph*). Uzly grafu představují samostatné, specializované výpočetní kroky či izolované agenty, zatímco orientované hrany určují pořadí závislostí a tok kontextových informací. V moderních orchestrátorech a CI/CD platformách (zejména v GitHub Actions) se závislosti mezi úlohami deklarují pomocí direktivy `needs: [...]`.

Jak podrobně rozvádí praktická část této práce na architektuře systému DarkFactory, celý životní cyklus automatizovaného požadavku je strukturován právě jako DAG složený z pěti klíčových fází:

1. **Detekce a kontext:** rozpoznání prostředí projektu, načtení konfiguračního souboru `darkfactory.json` a příslušných systémových pravidel.
2. **Interpretace a plánování:** formulace přesného technického postupu a vytvoření plánovacího úkolu dříve, než dojde k samotnému zásahu do kódu.
3. **Implementace (kódování):** spuštění agenta s přesně vymezenou sadou nástrojů v izolované větvi repozitáře.
4. **Verifikace a testování:** automatické sestavení projektu, spuštění testovací sady a kontrola kvality změn.
5. **Schvalovací brána (Governance):** podmíněné zastavení toku grafu a vyžádání lidské kontroly před finálním sloučením pull requestu.

Zásadní předností grafového uspořádání je determinismus a striktní bezpečnost: pokud kterýkoli uzel selže (např. automatizované testy detekují regresi), exekuce se v dané větvi grafu okamžitě přeruší a navazující kroky se nespustí, což zabráňuje poškození hlavní vývojové linie.

2.4 Human in the loop neboli člověk ve smyčce

Čím je systém samostatnější, tím důležitější je otázka, kde do procesu vstupuje člověk. Úplná autonomie není cílem; cílem je autonomie v rutinních krocích a lidské rozhodnutí tam, kde je nevratné nebo kde chybí měřítko správnosti.

2.4.1 Schvalovací body

Schvalovací bod je místo, kde se proces zastaví a čeká na potvrzení. Jeho umístění je kompromisem: příliš mnoho schvalování popírá smysl automatizace, příliš málo znamená ztrátu kontroly. Osvědčeným řešením je schvalovat *záměr* (co se má udělat) a *plán* (jak se to má udělat) dříve, než vznikne kód.

2.4.2 Dohledatelnost

Aby byla automatizovaná změna přezkoumatelná, musí být zřejmé, z jakého požadavku vzešla. Uchování doslovného znění původního zadání je proto součástí návrhu, nikoli formalitou: parafráze ztrácí význam, který do zadání vložil ten, kdo je formuloval.

2.4.3 Eskalace a transparence selhání

Systém, který své vlastní selhání zamlčí nebo zamete pod koberec, je nebezpečnější než systém, který zjevně havaruje: nespolehlivost se v něm stává neviditelnou a postupně eroduje důvěru uživatele v celý autonomní provoz. V konceptu člověka ve smyčce proto hlášení chyb a eskalace selhání netvoří pouhý doplňkový technický detail, nýbrž fundamentální bezpečnostní pilíř. Pokud autonomní agent narazí na vyčerpání kontextového okna, syntaktickou chybu neřešitelnou v rámci rozpočtu tahů nebo selhání integračních testů, nesmí skončit tichým uváznutím či nekonečným cyklem. Místo toho musí harness deterministicky zachytit chybový stav, sestavit strukturovaný diagnostický protokol (obsahující chybový stack trace, diff provedených změn a stav kontextu) a srozumitelně jej eskalovat člověku formou notifikace či dedikovaného incidentu. Tím je zajištěna plná observabilita a okamžitá lidská dohledatelnost.

3 Praktická část

Praktickou částí práce je systém **DarkFactory** — sada pravidel, skriptů a pracovních postupů, které z repozitáře udělají samostatně pracující vývojový provoz. Zdrojový kód je veřejně dostupný (11).

3.1 Cíl a rozsah systému

Systém má převzít rutinní kroky vývojového procesu: přijetí požadavku, jeho interpretaci, naplánování, provedení změny a její ověření. Nemá nahradit rozhodování o tom, co se má stavět; to zůstává člověku, a systém je navržen tak, aby si toto rozhodnutí vyžádal dříve, než začne pracovat.

3.2 Architektura

 **Strukturální upozornění: Architektura v rekonstrukci:** Vzhledem k probíhající zásadní přestavbě systému DarkFactory byla dosavadní implementační dekompozice modulů a skriptů dočasně nahrazena zástupnými strukturálními bloky. Nová architektura, komponentní diagram a specifikace rozhraní budou doplněny po stabilizaci nové verze.

Systém DarkFactory je navržen jako modulární stavebnice složená z řídicích orchestračních modulů, šablon pracovních postupů pro kontinuální integraci a hermetického^{*} běhového prostředí. Architektura striktně odděluje deklarativní konfiguraci konkrétního repozitáře od samotné logiky orchestrace.

3.2.1 Koncepce a systémové vrstvy

 **Návrh na vylepšení:** Placeholder: Zde bude doplněna nová vrstevnatá dekompozice přestavěného systému DarkFactory včetně nového komponentního schématu nahrazujícího původní skriptové jádro.

3.2.2 Sdílení pracovních postupů

 **Návrh na vylepšení:** Placeholder: Zde bude popsán nový distribuční a konzumační model workflow napříč spravovanými repozitáři po dokončení přestavby.

3.2.3 Konfigurační model repozitáře

 **Návrh na vylepšení:** Placeholder: Zde bude specifikováno nové deklarativní schéma konfiguračního manifestu vymezujícího identitu repozitáře, doménové oblasti a projektová pravidla.

^{*}Pojem **hermetické prostředí** (angl. *hermetic environment*) označuje v softwarovém inženýrství takové výpočetní a běhové prostředí, které je zcela izolované od nekontrolovaných stavů hostitelského operačního systému a okolní sítě. Veškeré nástroje, knihovny a systémové závislosti jsou v něm explicitně deklarovány a uzamčeny na konkrétních verzích, což zaručuje absolutní determinismus a reprodukovatelnost: proces spuštěný v hermetickém kontejneru skončí vždy identickým výsledkem bez ohledu na to, kde a kdy byl vyvolán.

3.3 Životní cyklus požadavku

🔧 **Strukturální upozornění: Životní cyklus v rekonstrukci:** Stavový automat řízení požadavků, definice exekučních fází a systém validačních bran procházejí revizí v návaznosti na celkovou přestavbu DarkFactory.

Životní cyklus požadavku formalizuje přechody mezi stavy od zadání úkolu v GitHub Issues přes plánování, implementaci, automatické testování až po vystavení pull requestu a sloučení do hlavní větve.

💡 **Návrh na vylepšení: Placeholder:** Zde bude doplněn nový stavový diagram životního cyklu a detailní popis jednotlivých fází nového běhového automatu.

3.4 Popis prostředí repozitáře

Aby systém věděl, které úlohy má spustit, musí rozpoznat, z čeho se repozitář skládá. Rozpoznávání vychází z názvů souborů popisujících balíček; jejich přítomnost je spolehlivější než jakýkoli ruční záznam, protože se nemůže rozejít se skutečností.

Soubor	Prostředí	Doména
pyproject.toml	Python	kód
Cargo.toml	Rust	kód
package.json	Node	kód
go.mod	Go	kód
typst.toml	Typst	text
.latexmkrc	LaTeX	text
lakefile.toml	Lean	matematika

Tabulka 1: Rozpoznávání prostředí podle souboru popisujícího balíček.

3.4.1 Domény a prostředí

Rozlišení dvou úrovní se ukázalo jako nutné až v průběhu práce. *Prostředí* říká, jaký nástroj je potřeba; *doména* říká, jakému způsobu řízení výsledek podléhá. Python a Rust jsou dvě prostředí téže domény — kód se testuje a balí. Typst a LaTeX jsou dvě prostředí jiné domény — text se sází a publikuje.

Toto rozlišení dovolu je popsat i repozitář, který obsahuje zároveň program a text — například projekt kombinující aplikační logiku se sazební dokumentací. Bez něj by bylo nutné buď považovat sazbu za zvláštní případ kódu, nebo pro texty vytvářet zcela samostatný orchestrační systém.

3.4.2 Deklarace jako doplněk rozpoznávání

Rozpoznávání prostředí nemůže předvídat veškeré specifické požadavky projektu. Pokud projekt například přibaluje vlastní sadu písem v podadresáři `fonts/` nebo specifické systé-

mové knihovny, příkaz pro sestavení vyžaduje dodatečné parametry. Konfigurační manifest proto umožňuje výchozí parametry transparentně rozšířit nebo přepsat, aniž by bylo nutné vypnout automatické rozpoznávání prostředí jako celek.

 **Návrh na vylepšení:** Placeholder: Zde bude uvedena ukázka deklarativního přepsání parametrů sestavení v novém schématu konfigurace přestavěného systému DarkFactory.

3.5 Orchestrace napříč poskytovateli

 **Strukturální upozornění: Orchestrační jádro v rekonstrukci:** Mechanismus předávání štafety, řízení kontextového okna a integrace klientských harnessů budou aktualizovány podle nového orchestračního enginu.

Systém neváže na jednoho poskytovatele modelu. Úkol je popsán způsobem nezávislým na rozhraní a předán prvnímu dostupnému poskytovateli; vyčerpá-li tento kvótu, předá se tentýž úkol dalšímu v pořadí, místo aby se proces zastavil. Tato vlastnost byla přímou reakcí na provozní zkušenost: zastavení celého procesu kvůli vyčerpané kvótě jediného poskytovatele bylo nejčastější příčinou prostoje.

 **Návrh na vylepšení:** Placeholder: Zde bude doplněn detailní popis nového modelu rotace, správy tokenů, bezpečnostního oddělení tajemství a bezeztrátové serializace kontextu po přestavbě systému.

3.6 Ověřování změn

 **Strukturální upozornění: Verifikační pipeline v rekonstrukci:** Způsob deterministického vyhodnocování integračních úloh a zpracování chybových stavů bude přizpůsoben nové architektuře.

Každá změna musí projít požadovanými kontrolami. Ty jsou navrženy tak, aby vždy skončily nějakým výsledkem — úloha, která se může „přeskočit“, by jinak zablokovala slučování napořád, jak bylo vysvětleno v kapitole 2.

 **Návrh na vylepšení:** Placeholder: Zde bude popsána nová podoba verifikačních úloh, cachování závislostí a automatického reportování incidentů po přestavbě systému.

3.7 Projektová automatizace a správa nastavení jako kód

 **Strukturální upozornění: Projektová automatizace v rekonstrukci:** Nástroje pro synchronizaci GitHub Projects, nastavení repozitáře a ochranu větví procházejí rekonstrukcí.

Automatizace repozitáře zajišťuje auditovatelnost a konzistentní správu projektových pravidel bez nutnosti manuálních zásahů v rozhraní platformy.

 **Návrh na vylepšení:** Placeholder: Zde bude popsána nová implementace projektové synchronizace a správy repozitářových pravidel jako kód v přestavěném systému DarkFactory.

3.8 Automaticky generovaná dokumentace

 **Strukturální upozornění: Dokumentační subsystém v rekonstrukci:** Generování a validace živé dokumentace budou přizpůsobeny novému rozhraní systému DarkFactory.

V moderním softwarovém vývoji představuje manuální údržba dokumentace permanentní zdroj chyb a desynchronizace: jakmile se kód vyvíjí rychleji než textové popisy, dokumentace se nevyhnutelně stává zastaralou a nespolehlivou. Systém DarkFactory proto prosazuje striktní princip stoprocentního odvozování veškeré projektové a API dokumentace přímo ze zdrojového kódu a strukturovaných inline komentářů (tzv. *living documentation*).

 **Návrh na vylepšení:** Placeholder: Zde bude doplněno nové procesní schéma a popis generování a striktní validace živé dokumentace v CI pipeline.

3.9 Systém revizních značek pro lidský dohled nad akademickým textem

Při rozšiřování systému DarkFactory na tvorbu a revizi akademických a odborných textů (doména textu) vyvstala potřeba formalizovat spolupráci člověka a autonomního agenta přímo v sazebním formátu Typst. Výsledkem je protokol vizuálních revizních značek (**Review Markers**) a textových revizních funkcí, který barevně a sémanticky rozlišuje stav zpracování jednotlivých pasáží:

 **Návrh na vylepšení:** Konstruktivní doporučení, nápady na rozšíření, doplnění schémat či návrhy na praktické propojení. Po zapracování se panel smaže.

 **Chyba / Nesrovnalost k opravě:** Detekované věcné nepřesnosti, logické mezery, překlepy nebo duplicita obsahu. Značka přesně formuluje vadu a zaniká s jejím odstraněním.

 **Strukturální upozornění:** Upozornění na hloubkovou nevyváženost kapitol, chybějící dekompozice komponent či nesoulad s osnovou práce.

 **Hloubková kritika / Oponentura:** Hloubková teoretická a architektonická oponentura bez servítků — odhalování slepých míst, neověřených předpokladů, bezpečnostních rizik a metodologických slabín formulovaných jako břitké otázky k obhajobě.

 **Metodické vymezení / Rozsah práce:** Metodické vymezení a rozsah práce — formulace hranic zkoumaného problému, metodická abstrakce (např. oddělení agentního inženýrství od strojového učení) a mantinely zadání.

V toku textu se uplatňují tyto zvýrazňovací a srovnávací funkce:

- **Žluté zvýraznění** (`#draft[...]` / `#unconfirmed[...]`): Označuje neověřený text konceptu čekající na autorské posouzení a revizi.
- **Zelené zvýraznění** (`#added[...]`): Označuje nově přidaný text vygenerovaný autonomním agentem na základě požadavku či doporučení.
- **Modré zvýraznění** (`#confirmed[...]`): Označuje text potvrzený uživatelem, který dosud neprošel finální integrací.
- **Červené zvýraznění s přeškrtnutím** (`#removed[...]`): Označuje text navržený k odstranění z rukopisu.
- **Původní nahrazovaný text** Srovnávací diff (`#diff(old, new)`): Zobrazuje původní text přeškrtnutý v červené barvě následovaný novým textem v zelené barvě.
- Čistý neoznačený text představuje finální, autorsky schválený a přijatý text v hlase autora (v rozpracovaném stavu konceptu jsou veškeré dosud neuzavřené pasáže zviditelněny revizními funkcemi).

Tento protokol umožňuje autonomnímu agentovi navrhopvat změny s transparentním významem míry jistoty a člověku poskytuje okamžitou vizuální kontrolu nad tím, které části rukopisu již prošly lidskou redakcí a které ještě čekají na posouzení.

4 Výsledky a diskuse

📌 **Metodické vymezení / Rozsah práce: Metodické vymezení výsledků a diskuse:** V souladu se zaměřením práce na principy agentního inženýrství a architekturu řídicího harnessu se tato kapitola soustředí na systémové a architektonické vlastnosti navrženého řešení: modularitu, determinismus, bezpečnostní izolaci a srovnání s existujícími přístupy (monolitické agentní smyčky vs. řízené DAG workflow).

4.1 Sjednocení pracovních postupů v architektuře harnessu

Základním architektonickým principem navrženého harnessu je centralizované sdílení pracovních postupů namísto jejich ad-hoc kopírování do jednotlivých projektů. Klientské projekty neudržují vlastní izolované sady validačních skriptů, linterů ani integračních definic; namísto toho delegují exekuci na centrální znovupoužitelné šablony a veškerou projektovou specifičnost deklarují v jediném manifestu.

Skutečný inženýrský přínos sjednocení nespočívá v pouhé kvantitativní redukci řádků kódu (LOC), nýbrž v kvalitativních a architektonických vlastnostech:

- **Údržbová složitost $O(1)$:** Před sjednocením vyžadovala jakákoli oprava či bezpečnostní aktualizace v CI procesu samostatnou manuální úpravu a pull request v každém projektu zvlášť ($O(N)$). V centralizovaném modelu je úprava provedena pouze jednou v řídicím harnessu a spotřebitelské projekty ji přebírají posunem připnuté verze v manifestu.
- **Statická kontrola a testovatelnost:** Centralizované jádro harnessu podléhá striktní typové kontrole a je pokryto sadou jednotkových testů.
- **Omezení jediného bodu selhání (SPOF):** Riziko nechtěných či nekompatibilních změn je deterministicky eliminováno verzováním: projekty odkazují na neměnný kryptografický SHA hash commitu či sémantický tag.
- **Konzistentní agentní prostředí:** Zajištění, že životní cyklus požadavku, správa kontextu, rotace poskytovatelů i vyhodnocování validačních bran probíhají ve všech projektech exaktně stejným způsobem.

4.2 Přenositelnost napříč vývojovými doménami

Klíčovou vlastností modulární architektury je striktní oddělení domény od prostředí. Zároveň prostředí specifikuje konkrétní sadu exekučních binárek (např. Python, Rust, Typst), doména vymezuje způsob řízení a verifikace výstupu:

- **Doména programového kódu:** Výstup podléhá kompilaci, statické analýze a exekuci jednotkových testů s měřením pokrytí.
- **Doména textu a dokumentace:** Výstup podléhá sazební kompilaci, kontrole terminologie a generování statických auditovatelných artefaktů (např. PDF dokumentace).

Díky deklarativnímu konfiguračnímu modelu dokáže řídicí harness obsluhovat obě domény identickým orchestračním automatem, aniž by bylo nutné duplikovat integrační infrastrukturu či větvit agentní smyčky.

4.3 Provozní metriky a spolehlivost agentních běhů

 **Strukturální upozornění: Metriky čekají na novou evaluaci:** Původní ad-hoc měření byla vyřazena. Po dokončení a stabilizaci nové verze harnessu proběhne série strukturovaných měření nad reálnými vývojovými požadavky.

 **Návrh na vylepšení:** Placeholder: Zde bude zařazena nová tabulka empirických metrik (počet zpracovaných požadavků, úspěšnost PR na první pokus, úspěšnost po automatické samoopravě, průměrná spotřeba tokenů na tah, časová latence a spolehlivost rotace modelů).

4.4 Poznatky a systémová úskalí z provozu

Praktické zkušenosti s během autonomních agentů v CI odhalily, že nejkritičtější úskalí neleží v neschopnosti modelů generovat syntakticky správný kód, nýbrž v distribuovaném řízení a systémové orchestraci:

- **Řízení souběžnosti a zámky:** Koordinace volajících a volaných úloh vyžaduje striktní hierarchizaci a zámky na úrovni větví, aby nedocházelo k uváznutí workflow nebo kolizím při souběžných integračních bězích.
- **Transparentnost hlášení selhání:** Systém nesmí tlumit výjimky a maskovat chyby; každé selhání API nebo nástroje musí být transparentně zaznamenáno v kontextu a eskalováno. Tiché maskování chyb vytváří iluzi stability a vede k rozsáhlým halucinacím modelu.
- **Dynamická detekce prostředí:** Systém nesmí spoléhat na implicitní předpoklady o technologiích v repozitáři, ale odvozovat exekuční plán z deklarativních manifestů.

4.5 Diskuse: Porovnání architektury

Zásadní otázkou je, jak si navržená architektura řídicího harnessu stojí ve srovnání se současnými agentními vývojovými platformami (např. SWE-agent (4), Devin či GitHub Copilot Workspace):

- **Deterministický DAG vs. nekonečná agentní smyčka:** Systémy jako SWE-agent spouštějí jediný monolitický model v interaktivní terminálové smyčce, kde model sám rozhoduje o ukončení práce. Pokud model uvízne v bludném kruhu nebo halucinuje, snadno vyčerpá celý rozpočet tokenů. Navržený harness naproti tomu uzavírá model do deterministického orientovaného acyklického grafu (DAG): každý krok (plánování, implementace, verifikace) má striktně alokovaný rozpočet tahů a samostatnou kontextovou izolaci.

- **Granularita lidského dohledu:** Většina komerčních nástrojů staví na paradigmatu „jedno zadání → finální pull request“. Pokud agent na začátku špatně pochopí záměr, vygeneruje stovky řádků nepoužitelného kódu, jehož následná revize vývojáře vyčerpává (*review fatigue*). Harness toto riziko eliminuje vícefázovým schvalováním (Human Gate): člověk autorizuje záměr a technický plán dříve, než je spuštěn samotný kódovací agent.
- **Bezpečnost a hermetičnost:** Běh autonomního agenta přímo na nechráněném vývojovém stroji přináší riziko poškození konfigurace nebo úniku tajemství. Navržený harness prosazuje exekuci v efemérních kontejnerech CI platformy s oddělenými oprávněními tokenů a auditovatelným protokolem každého provedeného příkazu.

4.6 Omezení

Dosažená zjištění je třeba interpretovat s ohledem na tři fundamentální omezení:

1. **Hranice deterministické verifikovatelnosti:** Systém vyžaduje, aby bylo správnost navržené změny možné objektivně ověřit automatizovanými testy, lintery či kompilátorem. U úloh subjektivní či kreativní povahy — jako je ergonomie uživatelského rozhraní, ladění vizuálních stylů nebo stylistická formulace odborného textu — zůstává autonomní přínos omezen na vygenerování prvotního návrhu, jehož validaci musí provést člověk.
2. **Propustnost a dostupnost inferenčních API:** Ačkoli rotační žebříček poskytovatelů minimalizuje dopad výpadku jedné služby, celková průchodnost pipeline je limitována globálními kvótami a latencí cloudových API. Při souběžném zpracování většího množství požadavků může dojít k vyčerpání všech dostupných účtů, což pipeline dočasně pozastaví.
3. **Absence vícevláknového řešení konfliktů:** Architektura se soustředí na deterministické řešení izolovaných požadavků. Řízení komplexních merge konfliktů a vzájemných interferencí při souběžné práci velkého množství lidských vývojářů a autonomních agentů nad stejnými větvemi zůstává otevřenou výzvou pro navazující výzkum.

5 Závěr

📌 **Metodické vymezení / Rozsah práce: Poznámka k vypracování závěru:** Závěr práce bude sepsán jako poslední krok po definitivním ucelení teoretických východisek agentního inženýrství a detailní architektury řídicího harnessu. Tato závěrečná kapitola syntetizuje zjištění o deterministickém řízení autonomních agentů a zhodnotí formulované principy a výzkumné otázky bez vazby na dřívější ad-hoc testovací repozitáře.

KONCEPT

Seznam příloh

A	Obsah přiloženého média	34
B	Schéma konfiguračního manifestu darkfactory.json	35
C	Sdílené workflow pro GitHub Actions	36
D	Systémové prompty plánovacího a kódovacího agenta	37
E	Protokol revizních značek v sazebním systému Typst	38
E.1	Vizuální reprezentace jednotlivých prvků v sazbě	38

KONCEPT

A Obsah přiloženého média

Odevzdaný archiv obsahuje kompletní zdrojové soubory práce, sazební šablonu, řídicí skripty a zdrojový kód autonomního systému DarkFactory.

```
mono-OdbornaPrace/
├─ prace/                                # Rukopis a sazba odborné práce
│   └─ kapitoly/                         # Texty jednotlivých kapitol (01-uvod
až 06-prilohy)
│   └─ lib/odborna-prace.typ             # Sazební šablona, recenzní značky a
formátování GJKT
│   └─ img/                             # Vektorová procesní schémata (SVG) a
grafické podklady
│   └─ fonts/                           # Metricky shodná patková písma
(Caladea)
│   └─ bib/references.bib                # Bibliografická databáze citovaných
zdrojů
│   └─ scripts/                         # Pomocné a náhledové skripty
│   └─ main.typ                         # Hlavní řídicí dokument sazby
│   └─ metadata.typ                     # Údaje o autorovi, vedoucím a
anotace
│   └─ Makefile                         # Příkazy pro automatizovanou
kompilaci a kontrolu
├─ darkfactory/                         # Zdrojový kód systému DarkFactory
(submodul)
│   └─ .github/workflows/               # Sdílené šablony kontinuální
integrace
│   └─ src/                             # Výkonné orchestrační jádro systému
├─ out/prace.pdf                        # Výsledný vysázený tiskový dokument
v PDF
└─ README.md                           # Dokumentace a návod na reprodukci
prostředí
```

Výpis 1: Stromová adresářová struktura odevzdaného elektronického archivu a doprovodných repozitářů.

B Schéma konfiguračního manifestu darkfactory.json

 **Strukturální upozornění: Příloha v rekonstrukci:** Formální JSON schéma manifestu bude doplněno po stabilizaci nového schématu v přestavěném systému DarkFactory.

 **Návrh na vylepšení:** Placeholder: Zde bude uvedeno úplné JSON Schema vymezující validní syntaxi nového konfiguračního manifestu.

KONCEPT

C Sdílené workflow pro GitHub Actions

 **Strukturální upozornění: Příloha v rekonstrukci:** Definice sdílených a volajících workflow pro GitHub Actions budou aktualizována po dokončení přestavby orchestrátoru.

 **Návrh na vylepšení:** Placeholder: Zde budou uvedeny ukázky znovupoužitelného a volajícího workflow nové verze DarkFactory.

KONCEPT

D Systémové prompty plánovacího a kódovacího agenta

 **Strukturální upozornění: Příloha v rekonstrukci:** Systémové prompty pro plánovací a implementační fáze budou aktualizovány podle nových rolí a instrukčních šablon v přestavěném systému.

 **Návrh na vylepšení:** Placeholder: Zde budou uvedeny kompletní systémové prompty pro jednotlivé fáze životního cyklu požadavku.

KONCEPT

E Protokol revizních značek v sazebním systému Typst

Tato příloha uvádí referenční definici a použití vizuálních revizních značek pro řízení a dohled nad generovaným textem v sazebním formátu Typst.

```
#import "lib/odborna-prace.typ": note, issue, alert, critique, added,
draft, confirmed, diff

// --- 1. Panely na okraji textu (Callouty) ---
#note[Doplňte porovnání rychlosti kompilace mezi verzemi 0.1 a 0.2.]
#issue[Chybná signatura funkce: chybí povinný parametr timeout.]
#alert[Sekce postrádá shrnutí naměřených výsledků před diskusí.]
#critique[Metodologická absence baseline prokazující přínos nového
modulu.]
#blue-note[Metodické vymezení: rozsah práce a oddělení agentního
inženýrství od strojového učení.]

// --- 2. Textové revizní funkce (zvýraznění v toku textu) ---
#draft[Tento odstavec tvoří neověřený koncept čekající na schválení.]
#added[Nově vygenerovaná sekce automaticky začleněná agentem.]
#confirmed[Uživatelem zkontrolovaný text, který ještě nebyl finalizován.]
#removed[Zastaralý text navržený k odstranění.]
#diff[Původní chybné znění textu.][Nové opravené znění textu po revizi.]
```

Výpis 2: Ukázka zápisu a použití revizních značek a textových funkcí v jazyce Typst.

E.1 Vizuální reprezentace jednotlivých prvků v sazbě

Pro přehlednost jsou níže uvedeny reálné ukázky jednotlivých revizních panelů a textových funkcí v jejich finální vizuální podobě:

 **Návrh na vylepšení:** Ukázka zeleného panelu doporučení (`#note`): Konstruktivní návrh na vylepšení, doplnění diagramu nebo námět na architektonickou optimalizaci.

 **Chyba / Nesrovnalost k opravě:** Ukázka červeného panelu vady (`#issue`): Zjištěná faktická nesrovnalost, logická mezera, syntaktická chyba či překlep vyžadující opravu.

 **Strukturální upozornění:** Ukázka žlutého panelu strukturálního upozornění (`#alert` / `#struct-alert`): Hlubková nevyváženost podkapitol, nekonzistence osnovy či absence klíčových náležitostí práce.

 **Hlubková kritika / Oponentura:** Ukázka oranžového panelu oponentury (`#critique`): Břitká, nekompromisní oponentura — zpochybnění neověřených předpokladů, analýza slabin metodiky a příprava na otázky zkušební komise.

 **Metodické vymezení / Rozsah práce:** Ukázka modrého panelu metodického vymezení (`#blue-note` / `#scope-note`): Vymezení rozsahu práce, formulace mantinelů a architektonických abstrakcí.

Ukázky textových zvýrazňovacích a srovnávacích funkcí v toku odstavce:

- **Neověřený koncept** (`#draft` / `#unconfirmed`): Tento text představuje koncept čekající na posouzení autorem.
- **Nově přidáný text** (`#added`): Tato pasáž byla nově vygenerována autonomním agentem na základě požadavku.
- **Potvrzený text** (`#confirmed`): Text byl předběžně odsouhlasen uživatelem, čeká na finální začištění.
- **Navrženo k odstranění** (`#removed`): ~~Tato neaktuální věta je navržena k úplnému smazání z rukopisu.~~
- **Srovnávací diff** (`#diff`): ~~Původní chybné nebo nepřesné znění pasáže.~~ Nové přesné, fakticky a formálně ověřené znění pasáže.
- **Čistý neoznačený text:** Představuje finální, autorsky schválený text v hlase autora bez jakéhokoliv podbarvení.

Každá značka plní přesně vymezenou komunikační roli v procesu lidského schvalování: zatímco finální text zůstává zcela bez zvýraznění, veškeré neověřené pasáže konceptu jsou zřetelně žluté (`#draft`), nově přidané části zelené (`#added`), potvrzené části modré (`#confirmed`) a opravy zviditelněné přes srovnávací diff (`#diff`). Náměty, chyby, strukturální vady i britká kritika jsou navíc striktně separovány do barevných postranních panelů na okraji textu.

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
3. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
4. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
5. XIAO, Guangxuan, TIAN, Yuandong, CHEN, Beidi, HAN, Song a LEWIS, Mike. Efficient Streaming Language Models with Attention Sinks. *arXiv preprint arXiv:2309.17453*. Online. 2023. Available from: <https://arxiv.org/abs/2309.17453>
6. ZHANG, Zhenyu, SHENG, Ying, ZHOU, Tianyi, CHEN, Tianlong, ZHENG, Lianmin, CAI, Ruisi, SONG, Zhao, TIAN, Yuandong, RÉ, Christopher, BARRETT, Clark, WANG, Zhangyang a CHEN, Beidi. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 34661–34730. Available from: <https://arxiv.org/abs/2306.14048>
7. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
8. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
9. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
10. Meta Harness: End to end optimization of Agent Harnesses. *arXiv preprint arXiv:2603.28052*. Online. 2026. Available from: <https://arxiv.org/abs/2603.28052>
11. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
12. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>

13. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
14. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>

KONCEPT